Yujen Lan

📍 Location: Taipei | ✉️ Email: johnblue.git@gmail.com | ☎️ Phone: 0908517935

🔗 Github: https://github.com/JohnBlue-git | 🔗 LinkedIn: https://www.linkedin.com/in/yu-jen-lan/

Systems Software Engineer with deep BMC/software domain expertise — built and verified an eBPF-based runtime intrusion detection system (HTTPS-Guard) for OpenBMC. Extending a systems/backend engineering background into security and detection engineering.

Summary

- Software Engineer with 3+ years of experience building Linux-based system software and backend services in Modern C++ and Python, including deep hands-on work inside a real BMC/OpenBMC management-plane codebase (Redfish APIs, DBus services).
- Daily **systems-level debugging experience** with the issue classes at the core of memory-safety and concurrency security work: use-after-free, object-lifetime defects, resource-ownership bugs, and race conditions.
- Identified structural blind spots in an industry-standard validation tool (DMTF Redfish Validator) — it couldn't verify field presence or operation ordering, only formatting — and designed a **new validation engine** to close the gap. Now covers 41 resource types and ~690 checks across real hardware, replacing ~40 manually-maintained modules.
- **Built and verified HTTPS-Guard**, an eBPF-based runtime intrusion detection system for BMC management interfaces (uprobes + XDP, 8 detection rules, integrated with Redfish EventService for alerting) — demonstrated end-to-end on live QEMU hardware.

HIGHLIGHTED SECURITY & SYSTEMS PROJECTS

HTTPS-Guard

eBPF-Based Runtime Intrusion Detection for OpenBMC Management Interfaces
GitHub: github.com/JohnBlue-git/HTTPS-Guard

- Built a host intrusion detection system combining eBPF (uprobes on OpenSSL's SSL_write, optional XDP enforcement) with Redfish EventService to detect and block malicious HTTPS/Redfish management traffic in real time.
- Implemented 8 distinct detection rules (outdated TLS versions, payload anomalies, certificate-access violations, volumetric attack patterns) and verified them live on QEMU-based hardware.
- Delivered a production-oriented design: systemd integration, configuration management, event logging, and documented platform-specific constraints.
- Connected kernel-space instrumentation to a userspace C++/Boost.Asio daemon for asynchronous event processing.

QEMU-eBPF-Sandbox

Linux Runtime Instrumentation & eBPF Development Framework
GitHub: github.com/JohnBlue-git/QEMU-eBPF-Sandbox

- Built an isolated QEMU-based environment for eBPF development and experimentation, with automated build scripts for kernel and filesystem images.
- Implemented CO-RE (Compile Once, Run Everywhere) examples across three architectural approaches: procedural C (libbpf/BTF), coroutine-based Modern C++, and Boost.Asio-based production-style I/O.
- Demonstrated end-to-end interaction between kernel-space instrumentation (XDP filtering, syscall tracing, socket/cgroup filtering) and user-space services.

HTTPS_Notes
TLS / PKI & HTTPS Security Deep-Dive
GitHub: github.com/JohnBlue-git/HTTPS_Notes

- Authored a comprehensive technical guide covering cryptography fundamentals, TLS/SSL handshake mechanics, certificate/PKI tooling (OpenSSL), and HTTP protocol evolution (1.0–3).
- Documented real-world attack history (BEAST, CRIME, POODLE, Heartbleed, ROBOT) and translated each into concrete hardening checklists (protocol/cipher selection, OCSP stapling, mTLS, automated renewal).
- Grounded the guide in a real-world C++ case study by walking through production source code (OpenBMC bmcweb).

Technical Skills

Software Engineering: Software Architecture | API Design | OOP | Design Patterns | RAII | Concurrency & Asynchronous Programming | Performance Optimization | System-Level Debugging | Refactoring
Platform Management Ecosystem: eBPF | XDP | uprobes / kprobes | TLS / SSL / PKI | OpenSSL | Linux observability & tracing | Host intrusion detection concepts
Languages & Tools: C++, C, Python, Bash | CMake, Meson, CI/CD | Linux, Systemd | Git

Experience

Insyde Software | Software Engineer *July 2024 – Present*

- **Reduced BMC startup time by over 60%** through startup profiling, dependency analysis, and service initialization optimization.
- **Ported and maintained platform-specific features on OpenBMC/bmcweb**, while continuously tracking upstreamOpenBMC developments and architecture changes.
- **Designed a scalable OpenBMC feature integration workflow** that reduced migration effort from approximately month to week while eliminating patch maintenance overhead.
- **Modernized legacy platform-management services** into modular Modern C++ and DBus-based architectures, improving maintainability.
- **Refactored Redfish schemas and service implementations**, improving maintainability, readability, and runtime performance by approximately 30%.
- **Improved DMTF Redfish compliance and interoperability** by resolving service-validator findings and protocol compatibility issues.
  GitHub: https://github.com/JohnBlue-git/RedfishAutoYamlTestInterview
- **Built reusable utility libraries** for resource management, file operations, and lifecycle control using RAII and object-oriented design principles.
- **Resolved production stability issues** including service crashes, object lifetime defects, resource ownership problems, and concurrency-related issues.
- **Reorganized OpenBMC meta-layer** structures and multi-repository projects while preserving development history and improving maintainability.
- **Upgraded legacy back-end service** architecture to a lightweight reverse-proxy model, improving deployment flexibility and operational simplicity.
- **Simplified firmware update workflows** through unified API-driven multipart update operations, improving usability and deployment efficiency.
- **Lead development of an end-to-end Redfish validation platform** capable of validating GET, PATCH, POST, and OEM workflows beyond existing industry validation tools — covering 40+ resource types across ~690 tests and 10 platform configurations.
- Diagnose and resolve cross-layer issues spanning Redfish services, DBus services, platform middleware, Linux systems, networking, and firmware components.
- Collaborate with software and firmware teams to deliver platform-management features and improve overall system reliability.

- Author technical API documentation and developer guides for internal teams and OEM partners.

SinoPac Securities | Software Engineer *June 2023 – April 2024*

- Developed and maintained backend applications using C++ on Linux platforms.
- **Improved data-processing efficiency by 3 to 4 times** through algorithm optimization and workflow redesign.
- Built automated CI/CD pipelines integrating build validation and testing processes, reducing manual effort and improving software quality.

Education

National Cheng Kung University (NCKU)

Master Degree *in Systems and Naval Mechatronic Engineering (Year: 2021 GPA: 4.3)*

Bachelor Degree *in Systems and Naval Mechatronic Engineering (Year: 2018 GPA: 3.7)*

Open Source Projects

MultiFish

Experimental project exploring REST API architecture, backend service design, and modern Golang system development.
Github: https://github.com/JohnBlue-git/MultiFish

HowToSDBusPlus

Practical guide and implementation examples for modern C++ DBus service development, inter-process communication, and Linux system programming.
GitHub: https://github.com/JohnBlue-git/HowToSDBusPlus

GoVarlink

Implementation examples and experiments focused on lightweight IPC mechanisms and service communication.
GitHub: https://github.com/JohnBlue-git/GoVarlink

HowToYocto

Documentation and practical examples covering Linux build systems, software integration, and platform customization.
GitHub: https://github.com/JohnBlue-git/HowToYocto

HowToPLDM

Technical notes and implementation examples focused on protocol architecture and system integration concepts.
GitHub: https://github.com/JohnBlue-git/HowToPLDM